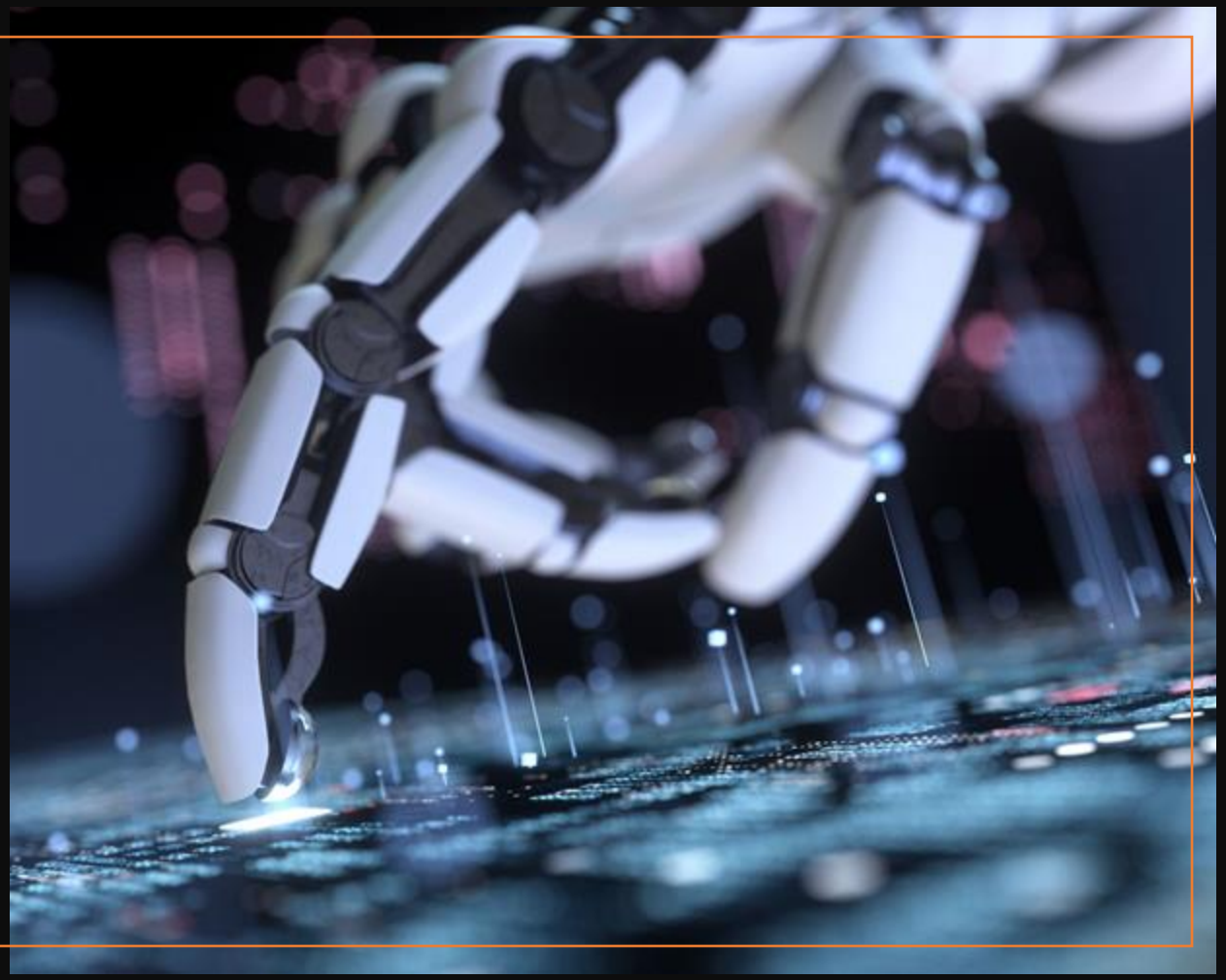
GenAI for CTFs

Leveraging Large Language Models and AI Tools for Cybersecurity Challenges



Session Agenda

- Introduction to LLMs and GenAI
- Prompt Engineering for CTF Success
- Understanding Model Context Protocol (MCP)
- Using Claude Desktop with MCPs for CTFs
- Hands-on Workshop
- Mini CTF Challenge

LLMs & GenAI Fundamentals

What are Large Language Models (LLMs)?

- AI models trained on vast amounts of text data
- Can understand and generate human-like text
- Examples: GPT-4, Claude, Gemini, LLaMA
- Trained to predict next words in sequences
- Can perform various tasks through natural language

Generative AI (GenAI) Overview

- AI that creates new content (text, code, images)
- Goes beyond simple pattern matching
- Can reason, analyze, and solve problems
- Multimodal capabilities (text, images, code)
- Interactive and conversational interfaces

Why GenAI for CTFs?

- Code analysis and reverse engineering assistance
- Cryptography problem solving
- Web vulnerability identification
- Binary analysis and exploitation
- Script generation and automation
- Learning and research acceleration

Prompt Engineering for CTFs

What is Prompt Engineering?

- Art and science of crafting effective AI prompts
- Maximizes LLM performance and accuracy
- Reduces ambiguity and improves results
- Essential skill for CTF problem-solving
- Involves context, instructions, and examples

Components of Effective CTF Prompts

- Role Definition: Set AI as security expert
- Context: Provide challenge background
- Specific Instructions: Clear, actionable tasks
- Constraints: Time, tools, approach limitations
- Output Format: How you want the response
- Examples: Show desired response style

System Prompt Template for CTFs

- You are an expert cybersecurity professional with 20+ years of experience in penetration testing, reverse engineering, and CTF competitions.

- For each challenge:
  - Analyze the problem systematically
  - Explain your thought process step-by-step
  - Provide multiple solution approaches
  - Include relevant commands/code
  - Highlight potential pitfalls
  - Suggest verification methods

Example: Web Security Prompt

- ROLE: You are a web application security expert
- CONTEXT: I'm analyzing a login page for SQL injection vulnerabilities
- TASK: Help me identify and exploit SQL injection flaws
- CONSTRAINTS: Use only manual testing techniques, no automated tools
- OUTPUT: Provide step-by-step testing methodology with specific payloads to try

Example: Binary Exploitation Prompt

- ROLE: You are a binary exploitation specialist
- CONTEXT: 32-bit Linux ELF binary with potential buffer overflow
- TASK: Analyze for vulnerabilities and develop exploit
- DATA: [Include objdump/strings output here]
- FORMAT: Provide analysis, vulnerability assessment, and exploit development steps with assembly explanations

Example: Cryptography Prompt

- ROLE: You are a cryptography expert and CTF player
- CONTEXT: Classical cipher challenge with encrypted text
- TASK: Identify cipher type and decrypt the message
- DATA: 'WKLV LV D WHVW PHVVDJH'
- APPROACH: Start with frequency analysis, consider common classical ciphers (Caesar, Vigenère, etc.)

Example: Prompt

- ROLE: You are a reverse engineering expert
- CONTEXT: Need to understand program logic to find hidden flag
- TASK: Analyze assembly code and identify flag extraction method
- DATA: [Include disassembly output]
- FOCUS: Explain control flow, identify key functions, and suggest dynamic analysis approaches

Advanced Prompting Techniques

- Chain of Thought: Ask for step-by-step reasoning
- Few-Shot Learning: Provide examples of similar problems
- Role Playing: Assign specific expert personas
- Constraint Setting: Define limitations and requirements
- Iterative Refinement: Build on previous responses
- Verification Requests: Ask AI to double-check work

Common Prompting Mistakes

- Being too vague or generic
- Not providing enough context
- Asking for direct answers without explanation
- Ignoring output format specifications
- Not setting appropriate constraints
- Forgetting to ask for verification steps

Prompt Engineering Best Practices

- Start with clear role definition
- Provide relevant context and data
- Be specific about desired output
- Include examples when possible
- Set appropriate constraints
- Ask for explanations, not just answers
- Iterate and refine based on results

Model Context Protocol (MCP)

What is Model Context Protocol?

- Open standard for connecting AI assistants to data sources
- Enables secure, controlled access to external tools
- Bridges the gap between LLMs and real-world applications
- Allows dynamic context injection
- Standardized way to extend AI capabilities

MCP Architecture

Client: AI application (like Claude Desktop)

Server: Provides tools and resources

Protocol: Standardized communication layer

Tools: Functions the AI can call

Resources: Data sources the AI can access

MCP Benefits for CTFs

- Direct file system access for challenge files
- Integration with security tools (nmap, burp, etc.)
- Database connectivity for data analysis
- Custom tool integration
- Real-time environment interaction

Claude Desktop + MCPs for CTFs

Setting Up Claude Desktop with MCP

- Install Claude Desktop application
- Configure MCP servers in settings
- Common MCP servers for CTFs:
  - File system access
  - Terminal/shell execution
  - Network tools integration
  - Custom security tool bridges

CTF Use Cases with Claude + MCP

- Binary analysis with direct file access
- Web challenge reconnaissance
- Cryptography puzzle solving
- Log analysis and forensics
- Reverse engineering assistance
- Exploit development guidance

Example: Binary Analysis Workflow

- Upload binary through MCP file server
- Run 'file' and 'strings' commands
- Analyze assembly with Claude's help
- Identify vulnerabilities or logic flaws
- Develop exploitation strategy
- Generate proof-of-concept code

Workshop Time!

Workshop Goals

Set up Claude Desktop with basic MCP

Practice effective prompt engineering

Apply prompts to common CTF scenarios

Learn iterative prompt refinement

Prepare for the mini CTF challenge

Workshop Exercise: Prompt Creation

1

Choose a CTF category (web, crypto, binary, forensics)

2

Create a system prompt for that category

3

Test with a sample challenge

Mini CTF Challenge

Tips for Using GenAI in CTFs

- Don't rely solely on AI - verify everything
- Use AI for learning and understanding
- Combine AI insights with manual analysis
- Ask for explanations, not just answers
- Practice prompt engineering skills
- Understand the limitations

Best Practices

- Always validate AI-generated code
- Use AI as a learning assistant
- Maintain hands-on technical skills
- Understand the 'why' behind solutions
- Practice without AI assistance too
- Stay updated with AI capabilities and limitations

Resources & Next Steps

- Claude Desktop: claude.ai/desktop
- MCP Documentation: modelcontextprotocol.io
- Prompt Engineering Guide: promptingguide.ai
- Practice platforms: HackTheBox, TryHackMe
- Join our community for more workshops!
- Follow cyber673 for updates